Informe de laboratorio 06

Tema: Programación Dinámica: Fundamentos y el Problema de la Diligencia

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
06	Programación Dinámica: Fundamentos y el Problema de la Diligencia	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	22/10/2026	26/10/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Diseña tu propio problema de la diligencia con **al menos 5 etapas**, y resuélvelo aplicando explícitamente cada paso de la técnica **SRTBOT** (documenta S, R, T, B, O y T en tu informe).

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <climits>
using namespace std;

int main() {

    const int n = 10;
    string nombres[n] = { "A", "B", "C", "D", "E", "F", "G", "H", "I", "J" };
    int etapa[n]      = {  1,   2,   2,   2,   3,   3,   3,   4,   4,   5  };

    vector<vector<pair<int, int>>> grafo(n);
    grafo[0] = { {1, 5}, {2, 3}, {3, 6} };
```

```
    grafo[1] = { {4, 4}, {5, 7}, {6, 6} };
    grafo[2] = { {4, 2}, {5, 5}, {6, 8} };
    grafo[3] = { {4, 6}, {5, 3}, {6, 4} };
    grafo[4] = { {7, 5}, {8, 7} };
    grafo[5] = { {7, 3}, {8, 4} };
    grafo[6] = { {7, 6}, {8, 2} };
    grafo[7] = { {9, 4} };
    grafo[8] = { {9, 5} };

    const int ORIGEN = 0, DESTINO = 9;

    vector<int> f(n, INT_MAX);
    vector<int> siguiente(n, -1);
    f[DESTINO] = 0;

    for (int u = DESTINO - 1; u >= ORIGEN; u--) {
        for (size_t k = 0; k < grafo[u].size(); k++) {
            int v     = grafo[u][k].first;
            int costo = grafo[u][k].second;

            if (f[v] != INT_MAX && costo + f[v] < f[u]) {
                f[u] = costo + f[v];
                siguiente[u] = v;
            }
        }
    }

    cout << "=== Tabla de la programacion dinamica (de J hacia A) ===\n";
    cout << "Etapa | Nodo | f(nodo) | Decision optima\n";
    for (int u = n - 1; u >= 0; u--) {
        cout << "  " << etapa[u] << "   |  " << nombres[u] << "   |   " << f[u] << "\t |  ";
        if (siguiente[u] == -1) cout << "-";
        else cout << nombres[u] << " -> " << nombres[siguiente[u]];
        cout << "\n";
    }

    cout << "\nCosto minimo desde A hasta J: " << f[ORIGEN] << "\n";

    cout << "Ruta optima: ";
    int actual = ORIGEN;
    while (actual != -1) {
        cout << nombres[actual];
        actual = siguiente[actual];
        if (actual != -1) cout << " -> ";
    }
    cout << endl;
    return 0;
}

```

- /relleno

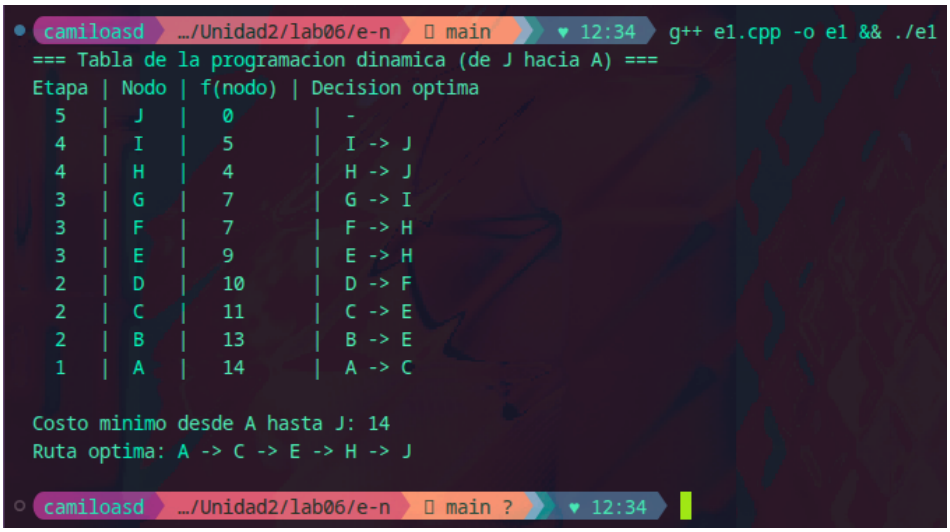


Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

Implementa **Fibonacci** de tres formas:

1. Recursivo ingenuo
2. Con memorización (top-down)
3. Con tabulación (bottom-up)

Compara sus tiempos de ejecución para $n = 10, 20$ y 30.

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
using namespace std;
using reloj = chrono::steady_clock;

long long llamadas = 0;

long long fibIngenuo(int n) {
    llamadas++;
    if (n <= 1) return n;
    return fibIngenuo(n - 1) + fibIngenuo(n - 2);
}

long long fibMemo(int n, vector<long long>& memo) {
    llamadas++;
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];
}

```

```cpp
long long fibTabulacion(int n) {
    if (n <= 1) return n;
    vector<long long> tabla(n + 1);
    tabla[0] = 0;
    tabla[1] = 1;
    for (int i = 2; i <= n; i++)
        tabla[i] = tabla[i - 1] + tabla[i - 2];
    return tabla[n];
}

template <typename F>
double medirMicros(F f, int repeticiones) {
    auto t0 = reloj::now();
    for (int i = 0; i < repeticiones; i++) f();
    auto t1 = reloj::now();
    double total = chrono::duration<double, micro>(t1 - t0).count();
    return total / repeticiones;
}

int main() {
    int valores[3] = { 10, 20, 30 };

    volatile long long sumidero = 0;

    cout << fixed << setprecision(3);
    cout << "Comparacion de tiempos (promedio, en microsegundos)\n";
    cout << "-----------------------------------------------------------------------\n";
    cout << " n | F(n)    | Ingenuo(us) [llamadas] | Memo(us) [llamadas] | Tabul.(us)\n";
    cout << "-----------------------------------------------------------------------\n";

    for (int idx = 0; idx < 3; idx++) {
        int n = valores[idx];

        int repIng = (n <= 10) ? 100000 : (n <= 20) ? 1000 : 5;
        llamadas = 0;
        long long resIng = fibIngenuo(n);
        long long llamIng = llamadas;
        double tIng = medirMicros([&]() { sumidero = fibIngenuo(n); }, repIng);

        llamadas = 0;
        { vector<long long> memo(n + 1, -1); fibMemo(n, memo); }
        long long llamMemo = llamadas;
        double tMemo = medirMicros([&]() {
            vector<long long> memo(n + 1, -1);
            sumidero = fibMemo(n, memo);
        }, 200000);

        double tTab = medirMicros([&]() { sumidero = fibTabulacion(n); }, 200000);

        vector<long long> m2(n + 1, -1);
        bool ok = (resIng == fibMemo(n, m2)) && (resIng == fibTabulacion(n));

        cout << setw(2) << n << " | " << setw(7) << resIng << " | "
             << setw(11) << tIng << " [" << setw(8) << llamIng << "] | "
             << setw(8) << tMemo << " [" << setw(3) << llamMemo << "] | "
             << setw(8) << tTab << (ok ? "   OK" : "   ERROR") << "\n";
```

```
    }
    cout << "--------------------------------------------------------------------------\n";
    cout << "Nota: el ingenuo crece ~1.6x por cada +1 en n (O(2^n)); memo y tabulacion crecen
    ↪  lineal.\n";
    return 0;
}

```

- /relleno

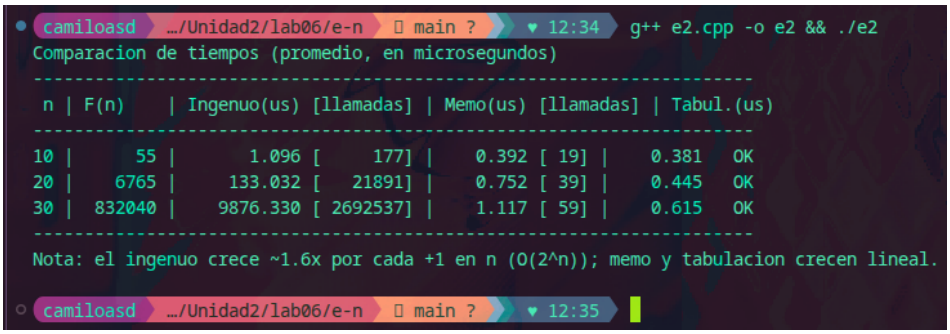


Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

Modifica el código del problema de la diligencia para que, cuando existan empates como el de D, se impriman **TODOS** los caminos óptimos encontrados, no solo uno.

```
#include <iostream>
#include <vector>
#include <string>
#include <climits>
using namespace std;

vector<vector<pair<int, int>>> grafo;
vector<int> f;
vector<vector<int>> optimos;
string nombres[] = { "A", "B", "C", "D", "E", "F", "G", "H" };
const int ORIGEN = 0, DESTINO = 7;
int totalCaminos = 0;

void enumerarCaminos(int u, vector<int>& camino) {
    camino.push_back(u);
    if (u == DESTINO) {
        totalCaminos++;
        cout << "  Camino " << totalCaminos << ": ";
        for (size_t i = 0; i < camino.size(); i++) {
            cout << nombres[camino[i]];
            if (i + 1 < camino.size()) cout << " -> ";
        }
        cout << "   (costo = " << f[ORIGEN] << ")\n";
    } else {
```

```
25
26          for (size_t k = 0; k < optimos[u].size(); k++)
27              enumerarCaminos(optimos[u][k], camino);
28      }
29      camino.pop_back();
30  }
31
32  int main() {
33      int n = 8;
34      grafo.assign(n, {});
35      grafo[0] = { {1, 2}, {2, 4}, {3, 3} };
36      grafo[1] = { {4, 7}, {5, 4}, {6, 6} };
37      grafo[2] = { {4, 3}, {5, 2}, {6, 4} };
38      grafo[3] = { {4, 4}, {5, 1}, {6, 5} };
39      grafo[4] = { {7, 3} };
40      grafo[5] = { {7, 6} };
41      grafo[6] = { {7, 3} };
42
43      f.assign(n, INT_MAX);
44      optimos.assign(n, {});
45      f[DESTINO] = 0;
46
47      for (int u = DESTINO - 1; u >= 0; u--) {
48          for (size_t k = 0; k < grafo[u].size(); k++) {
49              int v = grafo[u][k].first, costo = grafo[u][k].second;
50              if (f[v] == INT_MAX) continue;
51              int candidato = costo + f[v];
52
53              if (candidato < f[u]) {
54
55                  f[u] = candidato;
56                  optimos[u].clear();
57                  optimos[u].push_back(v);
58              } else if (candidato == f[u]) {
59
60                  optimos[u].push_back(v);
61              }
62          }
63      }
64
65      cout << "Costo minimo desde A hasta H: " << f[ORIGEN] << "\n\n";
66      cout << "Sucesores optimos por nodo (empate si hay mas de uno):\n";
67      for (int u = n - 2; u >= 0; u--) {
68          cout << "  f(" << nombres[u] << ") = " << f[u] << "  via ";
69          for (size_t k = 0; k < optimos[u].size(); k++)
70              cout << nombres[optimos[u][k]] << (k + 1 < optimos[u].size() ? ", " : "");
71          if (optimos[u].size() > 1) cout << "   <-- EMPATE";
72          cout << "\n";
73      }
74
75      cout << "\nTodos los caminos optimos:\n";
76      vector<int> camino;
77      enumerarCaminos(ORIGEN, camino);
78      cout << "\nTotal de caminos optimos: " << totalCaminos << endl;
79      return 0;
80  }
```

```
81
```

- /relleno

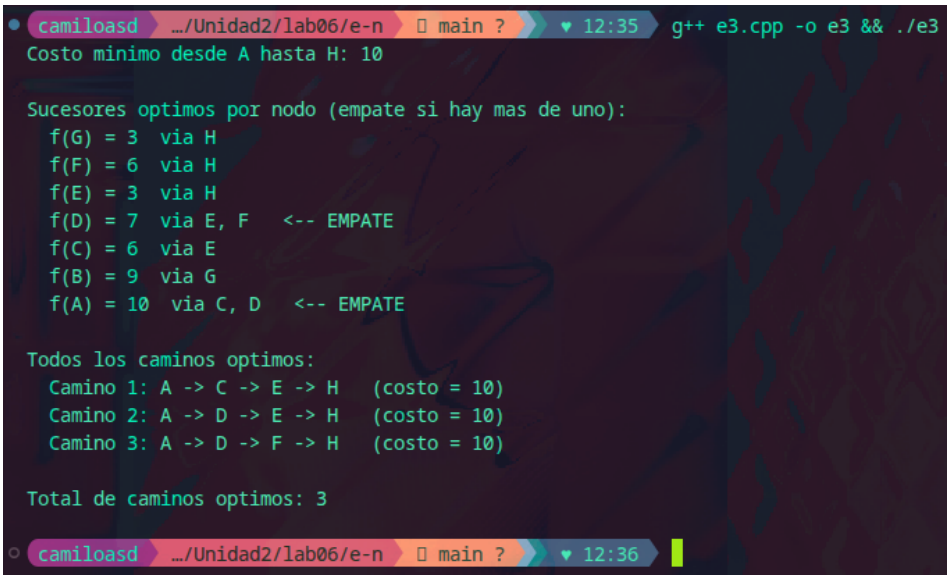


Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

Aplica la técnica **SRTBOT** para resolver un problema de camino de costo **MÁXIMO** (en vez de mínimo) sobre una malla de etapas similar a la del ejercicio resuelto.

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <climits>
using namespace std;

vector<vector<pair<int, int>>> grafo;
vector<int> f;
vector<vector<int>> mejores;
string nombres[] = { "A", "B", "C", "D", "E", "F", "G", "H" };
const int ORIGEN = 0, DESTINO = 7;
int totalCaminos = 0;

void enumerar(int u, vector<int>& camino) {
    camino.push_back(u);
    if (u == DESTINO) {
        totalCaminos++;
        cout << "  Camino " << totalCaminos << ": ";
        for (size_t i = 0; i < camino.size(); i++)
            cout << nombres[camino[i]] << (i + 1 < camino.size() ? " -> " : "");
        cout << "   (ganancia = " << f[ORIGEN] << ")\n";
```

```
    } else {
        for (size_t k = 0; k < mejores[u].size(); k++)
            enumerar(mejores[u][k], camino);
    }
    camino.pop_back();
}

int main() {
    int n = 8;
    grafo.assign(n, {});
    grafo[0] = { {1, 2}, {2, 4}, {3, 3} };
    grafo[1] = { {4, 7}, {5, 4}, {6, 6} };
    grafo[2] = { {4, 3}, {5, 2}, {6, 4} };
    grafo[3] = { {4, 4}, {5, 1}, {6, 5} };
    grafo[4] = { {7, 3} };
    grafo[5] = { {7, 6} };
    grafo[6] = { {7, 3} };

    const int NEG = INT_MIN;
    f.assign(n, NEG);
    mejores.assign(n, {});
    f[DESTINO] = 0;

    for (int u = DESTINO - 1; u >= 0; u--) {
        for (size_t k = 0; k < grafo[u].size(); k++) {
            int v = grafo[u][k].first, ganancia = grafo[u][k].second;
            if (f[v] == NEG) continue;
            int candidato = ganancia + f[v];
            if (candidato > f[u]) {
                f[u] = candidato;
                mejores[u].clear();
                mejores[u].push_back(v);
            } else if (candidato == f[u]) {
                mejores[u].push_back(v);
            }
        }
    }

    cout << "Ganancia maxima desde A hasta H: " << f[ORIGEN] << "\n\n";
    cout << "Tabla (de H hacia A):\n";
    for (int u = n - 1; u >= 0; u--) {
        cout << "  f(" << nombres[u] << ") = " << f[u];
        if (u != DESTINO) {
            cout << "   via ";
            for (size_t k = 0; k < mejores[u].size(); k++)
                cout << nombres[mejores[u][k]] << (k + 1 < mejores[u].size() ? ", " : "");
        }
        cout << "\n";
    }

    cout << "\nCaminos de ganancia maxima:\n";
    vector<int> camino;
    enumerar(ORIGEN, camino);
    cout << "\nTotal: " << totalCaminos << " camino(s)\n";
    cout << "(Para comparar: el camino de costo MINIMO de la guia vale 10.)\n";
    return 0;
```

```
78 }
79
```

- /relleno



Figura 4: Ejecución del ejercicio 4

1.5. Ejercicio 5

Investiga y documenta un problema real (logística, finanzas, biología computacional) que pueda modelarse como un problema de decisión multietapa similar al de la diligencia, y explica cómo se definirían sus subproblemas.

```cpp
#include <iostream>
#include <vector>
#include <climits>
#include <iomanip>
using namespace std;

int main() {

    const int T = 6;
    int d[T + 1] = { 0, 3, 2, 4, 1, 5, 2 };
    const int K = 10;
    const int c = 2;
    const int h = 1;
    const int IMAX = 8;
    const int XMAX = 8;
    const int INF = INT_MAX;

    vector<vector<int>> f(T + 2, vector<int>(IMAX + 1, INF));
```

```
    vector<vector<int>> prod(T + 2, vector<int>(IMAX + 1, -1));

    for (int i = 0; i <= IMAX; i++) f[T + 1][i] = 0;

    for (int t = T; t >= 1; t--) {
        for (int i = 0; i <= IMAX; i++) {
            for (int x = 0; x <= XMAX; x++) {
                int j = i + x - d[t];
                if (j < 0 || j > IMAX) continue;
                if (f[t + 1][j] == INF) continue;

                int costoMes = (x > 0 ? K : 0) + c * x + h * j;
                int total = costoMes + f[t + 1][j];
                if (total < f[t][i]) {
                    f[t][i] = total;
                    prod[t][i] = x;
                }
            }
        }
    }

    int inventario = 0;
    cout << "Costo minimo total de los " << T << " meses: " << f[1][inventario] << "\n\n";

    cout << "Plan optimo de produccion\n";
    cout << "Mes | Demanda | Inv.inicial | Producir | Inv.final | Costo del mes\n";
    cout << "----+---------+-------------+----------+-----------+--------------\n";
    int suma = 0;
    for (int t = 1; t <= T; t++) {
        int x = prod[t][inventario];
        int j = inventario + x - d[t];
        int costoMes = (x > 0 ? K : 0) + c * x + h * j;
        suma += costoMes;
        cout << setw(3) << t << " | " << setw(7) << d[t] << " | " << setw(11) << inventario
             << " | " << setw(8) << x << " | " << setw(9) << j << " | " << setw(12) <<
             ↪  costoMes << "\n";
        inventario = j;
    }
    cout << "Suma de los costos mensuales (verificacion): " << suma << endl;
    return 0;
}

```

- /relleno



Figura 5: Ejecución del ejercicio 5

2. Cuestionario

1. ¿Qué dos condiciones debe cumplir un problema para poder resolverse con Programación Dinámica?
2. ¿Cuál es la diferencia principal entre memorización y tabulación?
3. En la técnica SRTBOT, ¿qué garantiza específicamente el paso del orden topológico?
4. ¿Por qué el problema de la diligencia puede tener más de una solución óptima?
5. ¿Por qué aplicar Programación Dinámica a un algoritmo como Mergesort (Laboratorio 3) no traería ninguna mejora de eficiencia?

3. Conclusiones